COMSATS University, Islamabad Pakistan

MalTwin

By

Iman Fatima CIIT/SP23-BCT-021/ISB
Maaz Malik CIIT/SP23-BCT-025/ISB

Supervisor
Ms. Najla Raza

Bachelor of Science in Cyber Security (2023-2027)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

COMSATS University, Islamabad Pakistan

MalTwin

A project presented to
COMSATS Univeristy, Islamabad

In partial fulfilment
of the requirment for the degree of

Bachelor of Science in Cyber Security (2023-2027)

By

Iman Fatima CIIT/SP23-BCT-021/ISB
Maaz Malik CIIT/SP23-BCT-025/ISB

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be a reproduction of some other, we will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning

Iman Fatima

Maaz Malik

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CT) MalTwin was developed by **IMAN FATIMA (CIIT/SP23-BCT-021)** and **MAAZ MALIK (CIIT/SP23-BCT-025)** under the supervision of Ms.Naja Raza and that in her opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Cyber Security.

_____

Supervisor

_____

External Examiner

_____

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

Industrial Internet of Things (IIoT) environments deployed in critical infrastructure sectors including energy, manufacturing, water treatment, and transportation face an escalating threat from advanced malware campaigns. Traditional signature-based antivirus systems are fundamentally ineffective against polymorphic and obfuscated malware variants, while analyzing malware on live industrial hardware poses severe safety risks to physical equipment and operational processes. Publicly available IIoT-specific malware datasets are critically scarce and class-imbalanced, further limiting the training quality of detection models.

MalTwin addresses all three of these challenges through an integrated, AI-based framework combining Digital Twin simulation, binary-to-image visual analysis, and intelligent learning-based classification. A software-based Digital Twin environment is constructed using Docker containerization and Mininet network emulation, providing a fully isolated, realistic IIoT testbed in which malware can be safely executed and observed, emulating PLCs, sensors, MQTT brokers, and Modbus servers, without any risk to physical infrastructure.

Malware binaries are converted into grayscale images by reading their raw byte streams and reshaping them into 2D pixel matrices. These visual representations expose structural patterns, byte density distributions, entropy signatures, and code layout textures, that persist across obfuscation techniques and enable detection through computer vision methods. A data enhancement module applies image augmentation strategies to address dataset scarcity and class imbalance. Convolutional Neural Networks are trained on enhanced datasets to classify malware families and detect previously unseen variants.

An Explainable AI module using Grad-CAM generates heatmaps that identify which regions of the grayscale binary image drove classification decisions, providing analyst-interpretable explanations for every detection. An automated forensic reporting module generates PDF and JSON reports with MITRE ATT&CK for ICS tactic mapping, translating AI detection outputs into standardized threat intelligence language. The complete system is surfaced through an interactive Streamlit dashboard enabling file upload, visualization, detection, XAI display, and report generation in a unified browser-based interface.

MalTwin delivers a scalable, reproducible, and research-oriented framework that advances IIoT malware variant detection beyond the capabilities of any single existing approach, directly addressing identified gaps in the current research literature including the absence of safe IIoT simulation environments, XAI integration, and end-to-end pipelines for IIoT-specific malware analysis.

Abstract

The rapid integration of Industrial Internet of Things (IIoT) devices in critical infrastructure has improved automation and efficiency but has simultaneously exposed these systems to advanced and evolving malware threats that traditional signature-based defenses cannot effectively counter. Attacks such as Stuxnet and WannaCry demonstrate how malware variants can disrupt operational technology environments, causing physical damage, safety risks, and substantial economic losses. Three key challenges persist in the field: the inability of static signatures to detect obfuscated or polymorphic malware, the safety risks of analyzing malware directly on real IIoT hardware, and the scarcity and class imbalance of IIoT-specific malware datasets that limit detection model generalization.

This project proposes **MalTwin**, a Digital Twin-based framework that enables safe malware analysis through simulated IIoT environments combined with visual binary representation techniques. Executable files are transformed into grayscale images to expose structural patterns resilient to obfuscation, while data enhancement techniques are applied to mitigate dataset imbalance. Rather than relying on predefined static signatures, the framework employs intelligent learning-based classification using Convolutional Neural Networks to identify malware variants, including previously unseen samples. Explainable AI integration through Grad-CAM heatmaps provides transparent, analyst-interpretable explanations for each classification decision. An interactive Streamlit dashboard provides visualization, prediction results, and system monitoring. Automated forensic reporting with MITRE ATT&CK for ICS mapping translates detection outputs into actionable threat intelligence. MalTwin delivers a scalable, research-oriented prototype for IIoT malware variant detection, emphasizing safety, adaptability, interpretability, and robustness in resource-constrained industrial environments.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor Ms.Najla Raza. Without her personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are grateful to her for her encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us with the values of honesty & hard work

Iman Fatima

Maaz Malik

Abbreviations

IIoT	Industrial Internet of Things
AI	Artificial Intelligence
CNN	Convolutional Neural Network
XAI	Explainable Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
IT	Information Technology
OT	Operational Technology
PLC	Programmable Logic Controller
RTU	Remote Terminal Unit
SCADA	Supervisory Control and Data Acquisition
HMI	Human Machine Interface
PE	Portable Executable
ELF	Executable and Linkable Format
MQTT	Message Queuing Telemetry Transport
TCP	Transmission Control Protocol
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques and Common Knowledge
ICS	Industrial Control System
GUI	Graphical User Interface
SHA	Secure Hash Algorithm
SOC	Security Operation Center
FR	Functional Requirement
NFR	Non-Functional Requirement

Table of Contents

Abstract v
1 Chapter 1: Introduction and Problem Definition 1
1.1 Overview of the Project 1
1.2 Vision Statement 1
1.3 Problem Statement 2
1.4 Problem Solution 2
1.5 Objectives of the Proposed System 2
1.6 Scope 3
1.6.1 Limitations and Constraints 3
1.7 Modules 4
1.7.1 Module 1: Digital Twin Simulation 4
1.7.2 Module 2: Binary-to-Image Conversion 4
1.7.3 Module 3: Dataset Collection and Preprocessing 5
1.7.4 Module 4: Data Enhancement and Balancing 5
1.7.5 Module 5: Intelligent Malware Detection 5
1.7.6 Module 6: Dashboard and Visualization 6
1.7.7 Module 7: Explainable AI and Interpretability 6
1.7.8 Module 8: Automated Threat Reporting and Intelligence 6
1.8 Related System Analysis and Literature Review 7
1.8.1 Literature Review 7
1.8.2 Related System Analysis 8
1.9 Tools and Technologies 9
1.10 Project Contribution 10
1.11 Relevance to Course Modules 11
2 Chapter 2: Requirement Analysis 12
2.1 User Classes and Characteristics 12
2.2 Operating Environment 13
2.3 Design and Implementation Constraints 14
2.4 Use Case Diagram 15
2.4.1 Detailed Use Case Descriptions 17
2.5 Requirement Identifying Technique 20
2.5.1 Mockup-Based Requirement Analysis 20
2.6 Functional Requirements 20
2.6.1 Mockup M1, Main Dashboard Screen 21
2.6.2 Mockup M2, Digital Twin Monitor Screen 22
2.6.3 Mockup M3, Binary Upload and Visualization Screen 23
2.6.4 Mockup M5, Malware Detection and Prediction Screen 24
2.6.5 Event-Response Table, Backend Process Requirements 25
2.7 Non-Functional Requirements 26
2.7.1 Reliability 26
2.7.2 Usability 27
2.7.3 Performance 27
2.7.4 Security 28
2.8 External Interface Requirements 28
2.8.1 User Interface Requirements 28
2.8.2 Software Interfaces 29
2.8.3 Hardware Interfaces 29
2.8.4 Communications Interfaces 30

1 Chapter 1: Introduction and Problem Definition

This chapter introduces the MalTwin project, its background, domain, vision, problem statement, and proposed solution. It further defines the project scope, objectives, modules, related system analysis, tools and technologies, original contributions, and relevance to academic coursework. MalTwin is an AI-based framework for malware variant detection in Industrial Internet of Things (IIoT) environments, combining Digital Twin simulation, binary-to-image visual analysis, and intelligent learning-based classification.

1.1 Overview of the Project

MalTwin is an advanced malware detection and analysis framework designed to secure Industrial Internet of Things (IIoT) environments through the integration of digital twin technology and deep learning. As critical infrastructure sectors like energy grids and manufacturing plants increasingly converge their IT and OT systems, they become high-stakes targets for sophisticated cyber physical attacks. MalTwin addresses the urgent need for security in these environments by providing a risk free, simulated space to execute and analyze threats that would otherwise be too dangerous to study on live industrial hardware.

The project addresses a landscape where traditional defense mechanisms often fail because they rely on static signatures that cannot catch polymorphic or zero day threats. Furthermore, the high stakes of industrial operations make it impractical to analyze live malware on real hardware due to the risk of physical damage or operational shutdowns. MalTwin solves these challenges by utilizing a software based Digital Twin built with Docker and Mininet to create an isolated simulation for safe malware observation. It further enhances detection by employing a binary to image conversion pipeline that transforms executable files into grayscale images, allowing Convolutional Neural Networks to identify structural patterns that are resilient to common obfuscation techniques.
The proposed solution offers a scalable and automated interface while improving transparency and efficiency in industrial cybersecurity. By incorporating Explainable AI and mapping forensic results to the MITRE ATT&CK for ICS framework, MalTwin provides security analysts with an interactive Streamlit dashboard that simplifies the complex process of identifying and mitigating modern IIoT malware.

1.2 Vision Statement

For IIoT security researchers and practitioners who need robust detection of rapidly evolving malware variants beyond signature-based limitations, MalTwin is an AI-based malware detection framework that provides safe simulation via Digital Twins, visual binary analysis, generative augmentation, and deep learning classification. Unlike conventional tools relying on static signatures or risky dynamic analysis, our system delivers a modular, scalable research platform for identifying unseen threats in IIoT through intelligent pattern recognition and synthetic data robustness.

1.3 Problem Statement

Industrial IoT (IIoT) systems are vulnerable to rapidly evolving malware, as minor changes like obfuscation or polymorphism can bypass traditional signature-based defenses, seen in attacks such as WannaCry and Stuxnet. Analyzing malware on real IIoT hardware is unsafe and impractical, while datasets are scarce and imbalanced, limiting detection accuracy. Existing tools relying on static signatures or limited behavioral analysis fail against unseen variants. A secure, scalable framework is therefore needed to simulate IIoT environments, enhance data diversity, visualize threats, and accurately detect malware in resource-constrained industrial systems.

1.4 Problem Solution

MalTwin addresses the identified challenges by proposing an integrated AI-based framework that combines Digital Twin simulation with visual malware analysis and adaptive learning techniques. The system creates a virtual IIoT testbed where malware can be safely executed and observed without risking physical infrastructure. Executable files are transformed into grayscale images to capture intrinsic structural patterns that remain resilient against obfuscation and packing techniques.

To overcome dataset scarcity and imbalance, the framework supports flexible data enhancement strategies, enabling improved generalization across malware families. Classification and variant detection are performed using intelligent analytical models selected based on experimental evaluation. System performance is evaluated using standard metrics such as accuracy, precision, recall, F1-score, and confusion matrices. A user-friendly dashboard enables visualization, file analysis, and system monitoring.

1.5 Objectives of the Proposed System

BO-1: Develop a software-based Digital Twin to emulate IIoT network topologies, device behaviors, and industrial communication protocols (Modbus, MQTT) for safe and repeatable malware analysis without risk to physical infrastructure.

BO-2: Implement a complete binary-to-image conversion pipeline capable of transforming PE and ELF executable binaries into standardized grayscale images for visual structural analysis.

BO-3: Apply data enhancement and class-balancing techniques to IIoT malware datasets to address sample scarcity and class imbalance, improving model generalization across malware families and previously unseen variants.

BO-4: Design, implement, and experimentally evaluate intelligent learning-based classification models, including Convolutional Neural Networks, capable of detecting polymorphic, obfuscated, and zero-day malware variants through visual pattern recognition.

BO-5: Rigorously evaluate detection performance using standard metrics including accuracy, precision, recall, F1-score, and confusion matrices on both seen and unseen malware samples.

BO-6: Integrate Explainable AI (XAI) techniques, specifically Grad-CAM, to generate interpretable

heatmap explanations of classification decisions, making detection reasoning transparent and auditable for security analysts.

BO-7: Build an interactive Streamlit-based dashboard enabling file upload, binary-to-image display, real-time malware detection, confidence visualization, XAI heatmap display, and one-click forensic report generation.

BO-8: Implement automated forensic reporting in PDF and JSON formats with MITRE ATT&CK for ICS tactic mapping, providing actionable threat intelligence output for security operations teams.

1.6 Scope

The scope of MalTwin encompasses the comprehensive design, development, integration, and evaluation of a software based framework specifically engineered for malware variant detection within Industrial Internet of Things environments. The project is structured into eight interconnected modules that collectively facilitate the transition from raw binary data to actionable security intelligence. At its foundation, the Digital Twin simulation module constructs a virtual IIoT network using Docker containers and Mininet to emulate nodes such as PLCs, sensors, and MQTT brokers within an isolated environment. The binary to image conversion module then processes Portable Executable and Executable and Linkable Format files by transforming raw byte streams into standardized 128 by 128 grayscale images. To ensure robust training, the dataset module aggregates samples from repositories like Malimg and VirusShare while the data enhancement module applies augmentation techniques to address class imbalances. The core detection module utilizes learning based classification models to identify malware families, which are then interpreted through an Explainable AI module using Grad-CAM heatmaps to highlight decision relevant features. All findings are centralized within an interactive Streamlit dashboard and exported through an automated reporting module that maps threats to the MITRE ATT&CK for ICS framework. Explicitly excluded from this scope are the integration of physical IIoT hardware and the execution of automated remediation or quarantine actions. Furthermore, the project does not involve the creation of malware binaries or production grade hardening for enterprise deployment. The boundary of this solution is strictly defined for research, academic demonstration, and educational purposes within a controlled simulation.

1.6.1 Limitations and Constraints

LI-1: Computational Overhead of Explainable AI, The integration of Grad-CAM-based XAI explanations adds significant per-file processing overhead, as it requires a full backward pass through the classification network. In high-throughput environments, users may need to disable XAI generation to maintain near-real-time analysis performance.

LI-2: Susceptibility to Adversarial Image Perturbations, Because the detection model classifies malware based on visual patterns derived from grayscale binary images, it is theoretically vulnerable to adversarial attacks in which minimal modifications to the binary alter image texture without changing malicious behavior.

LI-3: Containerized Execution Environment Boundaries, The Digital Twin is strictly containerized, preventing realistic emulation of malware that requires kernel-level interactions, hardware-specific interrupt handling, or physical device drivers that cannot be virtualized within Docker.

LI-4: Dataset Generalization Limitations, Model training relies on publicly available malware datasets (primarily Malimg) dominated by Windows PE samples. Detection accuracy for rare IIoT-specific ELF malware families may be lower than for well-represented Windows PE families.

LI-5: Single-Node Research Deployment, The framework is designed for single-machine research deployment and does not support distributed model training or multi-machine pipeline execution without significant architectural modifications.

1.7 Modules

MalTwin is organized into the following functional modules

1.7.1 Module 1: Digital Twin Simulation

This module constructs and manages the software-based IIoT Digital Twin environment, providing the safe malware execution and behavioral observation capability that distinguishes MalTwin from purely static analysis approaches.

FE-1: Deploy and manage Docker containers representing distinct IIoT node roles including PLCs, sensors, MQTT broker, Modbus TCP server, and network gateway.
FE-2: Define and instantiate realistic IIoT network topologies using Mininet, including node connectivity, bandwidth constraints, and protocol-specific port assignments.
FE-3: Generate synthetic normal IIoT traffic including Modbus polling cycles and MQTT sensor telemetry for baseline behavioral profiling.
FE-4: Support controlled deployment of malware samples within isolated Docker containers and capture resulting network traffic for behavioral artifact analysis.

1.7.2 Module 2: Binary-to-Image Conversion

This module implements the core binary visualization pipeline, transforming raw executable binary files into standardized grayscale images for downstream visual analysis and classification.

FE-1: Accept PE (.exe, .dll) and ELF binary files as input and validate file format using PE/ELF header signature checks before processing.
FE-2: Read binary files as raw byte streams and reshape byte arrays into 2D numpy matrices using file-size-proportional width calculations.
FE-3: Render reshaped byte arrays as 128×128 grayscale PNG images using bilinear interpolation for standardized model input.
FE-4: Compute SHA-256 cryptographic hash of each uploaded binary for file identification and forensic audit trail purposes.

1.7.3 Module 3: Dataset Collection and Preprocessing

This module manages the acquisition, organization, validation, and preprocessing of malware and benign binary datasets required for model training and evaluation.

FE-1: Source malware image samples from the Malimg dataset, EMBER feature dataset, and IIoT-relevant sample repositories including VirusShare and IoT-23.
FE-2: Normalize grayscale pixel values to the range [0, 1] and encode class labels for multi-class classification compatibility.
FE-3: Perform stratified train/validation/test splitting ensuring proportional class representation in all three splits.
FE-4: Validate dataset integrity by verifying sample counts, label consistency, and absence of corrupted or duplicate image files.

1.7.4 Module 4: Data Enhancement and Balancing

This module addresses the dataset scarcity and class imbalance challenges by applying domain-adapted augmentation strategies to the grayscale binary image representation.

FE-1: Apply random rotation (up to ±15 degrees), horizontal and vertical flipping, and brightness adjustment to training images.
FE-2: Inject Gaussian noise into training images to improve model robustness against minor binary perturbations.
FE-3: Implement class-aware oversampling to ensure equitable representation of minority malware families in each training batch.
FE-4: Provide a dataset gallery visualization displaying sample images from each malware family class for quality inspection.

1.7.5 Module 5: Intelligent Malware Detection

This module implements, trains, and evaluates the deep learning classification models that form the core detection capability of the MalTwin framework.

FE-1: Implement Convolutional Neural Network architectures for multi-class classification of grayscale malware images.
FE-2: Train models on the enhanced, balanced dataset with configurable hyperparameters including learning rate, batch size, and number of training epochs.
FE-3: Evaluate trained models using accuracy, precision, recall, F1-score, and a full confusion matrix on held-out test data.
FE-4: Output per-class prediction probability distributions and a top-1 predicted malware family label with associated confidence score.
FE-5: Support serialization of trained model weights in PyTorch .pt format for reproducible inference.

1.7.6 Module 6: Dashboard and Visualization

This module provides the primary user interface of MalTwin through an interactive Streamlit web application, enabling security analysts and researchers to interact with all framework capabilities through a browser-based GUI.

FE-1: Provide a Streamlit-based main dashboard displaying system module status, cumulative detection statistics, and navigation to all system sections.
FE-2: Support binary file upload (PE and ELF formats, maximum 50 MB) with immediate display of the converted grayscale image, file metadata, and SHA-256 hash.
FE-3: Display real-time malware detection results including predicted family label, confidence percentage bar, and per-class probability distribution chart.
FE-4: Render Grad-CAM XAI heatmaps overlaid on grayscale binary images when explainability is requested by the user.
FE-5: Provide a dataset visualization gallery showing sample images from each malware family class.
FE-6: Display Digital Twin simulation status, active node count, and live traffic log within the dashboard.

1.7.7 Module 7: Explainable AI and Interpretability

This module provides transparent and auditable explanations for the detection model's classification decisions using gradient-based visual attribution methods, addressing the 'black box' criticism of deep learning in security-critical applications.

FE-1: Implement Gradient-weighted Class Activation Mapping (Grad-CAM) using the Captum library to generate class-discriminative localization maps for each classification decision.
FE-2: Overlay Grad-CAM heatmaps on the input grayscale binary image, highlighting the specific byte regions that most strongly influenced the classification outcome.
FE-3: Provide textual interpretation annotations alongside the heatmap, explaining which structural byte regions correspond to high-attribution areas and their potential significance.
FE-4: Support export of XAI heatmap images for inclusion in forensic reports.

1.7.8 Module 8: Automated Threat Reporting and Intelligence

This module transforms raw detection outputs into structured, actionable threat intelligence reports aligned with established cybersecurity frameworks.

FE-1: Automatically generate structured forensic reports in both PDF and JSON formats immediately following a detection event.
FE-2: Include in each report: file name, SHA-256 hash, file size, upload timestamp, predicted malware family, confidence score, and Grad-CAM heatmap image (if generated).
FE-3: Map each detected malware family to relevant MITRE ATT&CK for ICS tactics and techniques using a locally stored JSON reference database.
FE-4: Log all detection events, including timestamp, file hash, prediction result, and confidence, to a local SQLite database for historical trend analysis and audit compliance.

FE-5: Provide a detection history view within the dashboard allowing sorting and filtering of past analysis events.

1.8 Related System Analysis and Literature Review

This section surveys existing work in network intrusion detection, classical machine learning approaches, and the emerging field of quantum machine learning for cybersecurity. It identifies gaps that MalTwin addresses.

1.8.1 Literature Review

Research in malware detection has evolved along two primary trajectories relevant to MalTwin: visual binary analysis and simulation-based dynamic analysis. The foundational contribution of Nataraj et al. (2011) introduced the binary visualization approach through the Malimg dataset, demonstrating that malware families share consistent and distinctive grayscale visual textures arising from recurring code patterns, data structures, and cryptographic constants. Their k-NN classifier achieved competitive accuracy on 25 malware families, establishing visual analysis as a viable alternative to signature-based detection and inspiring a substantial body of subsequent research.

Building on this foundation, Saridou et al. (2023) applied alpha-cut thresholding and binary visualization techniques across diverse malware datasets, confirming the robustness of structural image features against obfuscation and achieving strong multi-class classification results. Transfer learning approaches have further extended the viability of visual malware analysis: a 2024 PMC review demonstrated that pre-trained vision models (VGG, ResNet, EfficientNet) can be fine-tuned on malware image datasets with significantly reduced training data requirements, directly addressing the dataset scarcity challenge that MalTwin also confronts.

Research specifically targeting IIoT environments has lagged behind the general malware detection literature. A 2020 study published in Ad Hoc Networks explored hybrid image visualization and deep learning models for malware detection in IIoT contexts, identifying the key challenges of protocol diversity, resource-constrained device profiles, and the absence of IIoT-specific training datasets. Cha et al. (2024) presented an intelligent anomaly detection system specifically combining malware image augmentation with Digital Twin simulation in an IIoT context, representing the closest prior work to MalTwin. However, their work focuses on anomaly detection rather than multi-class variant classification, does not incorporate explainable AI, and lacks automated threat reporting or MITRE ATT&CK mapping. The SoK paper on visualization-based malware detection (ACM, 2024) systematized the current state of the art, identifying the absence of IIoT-specific frameworks, XAI integration, and end-to-end pipelines as key open research gaps. MalTwin directly targets all of these identified gaps.

1.8.2 Related System Analysis

Table 1.1: Related System Analysis with Proposed Project Solution

Application Name	**Weakness**	**Proposed Project Solution**
Traditional Signature-Based Antivirus (Symantec, McAfee, ClamAV)	Completely ineffective against polymorphic variants, obfuscated binaries, and zero-day threats. Requires continuous signature database updates. Cannot detect structurally modified variants of known malware families.	Replaces signature matching with visual structural analysis. Grayscale image patterns remain stable across obfuscation transformations, enabling detection of variants that signature tools cannot identify.
Malimg Visualization Framework (Nataraj et al., 2011)	Static visualization only on general Windows PE malware. No simulation environment for safe IIoT-specific execution. No data augmentation. No XAI or reporting functionality.	Extends the visual approach with a Digital Twin for safe IIoT emulation, IIoT-relevant dataset sourcing, generative augmentation to address data scarcity, XAI heatmaps, and automated MITRE-mapped reporting.
General CNN/ML Malware Detection Frameworks (ResNet, VGG-based tools)	Target general Windows malware corpora. Lack IIoT protocol context, safe execution environment, and domain-adapted datasets. No interpretability or forensic output.	Provides an IIoT-focused framework integrating visual classification within a complete pipeline including safe Digital Twin simulation, dataset balancing, XAI, and structured forensic reporting.
Digital Twin ICS Monitoring Tools (Siemens MindSphere, AWS IoT TwinMaker)	Focused on operational monitoring and anomaly detection. Minimal or no integration with malware variant classification, visual binary analysis, or AI-driven detection pipelines.	Integrates Digital Twin simulation directly with visual malware analysis, deep learning classification, and XAI in a unified research framework specifically targeting malware variant detection.
Cha et al. (2024), IIoT Anomaly Detection with DT and Image Augmentation	Targets anomaly detection rather than multi-class variant classification. No XAI integration. No automated forensic reporting. No MITRE ATT&CK mapping. No Streamlit dashboard.	Extends this approach with multi-class variant classification, Grad-CAM XAI, automated PDF/JSON forensic reports, MITRE ATT&CK for ICS mapping, SQLite event logging, and an interactive analyst dashboard.

1.9 Tools and Technologies

Table 1.2: Tools and Technologies for the MalTwin Project

Tool/Technology	Purpose	Rationale
Docker	Containerization and isolation for Digital Twin IIoT simulation environments	Industry-standard container platform ensuring reproducible, isolated execution; native integration with Mininet for network emulation
Mininet	IIoT network topology emulation with Modbus and MQTT protocol support	Lightweight software-defined networking emulator supporting custom topologies; directly integrable with Docker for realistic IIoT simulation
Streamlit	Interactive web dashboard for all user-facing visualization and analysis interactions	Rapid Python-native UI development; ideal for data science dashboards with file upload, chart rendering, and real-time updates with no frontend code required
OpenCV	Image processing pipeline for binary-to-grayscale conversion and preprocessing	Comprehensive, performant computer vision library with robust image I/O, resizing, and preprocessing operations; GPU acceleration support
Pillow (PIL)	Lightweight image handling, format conversion, and display support	Simple and reliable API for image loading, saving, format conversion, and manipulation tasks complementing OpenCV
PyTorch	Primary deep learning framework for CNN model implementation, training, and inference	Flexible dynamic computation graph framework strongly preferred in research settings; native support for Captum XAI library
TensorFlow/ Keras	Alternative deep learning framework for comparative experimentation and ablation studies	Provides additional model architectures and Keras high-level API for rapid benchmarking of alternative classification approaches
NumPy	Numerical computation and byte-level array manipulation for binary-to-image pipeline	Foundational array processing library; direct byte array operations essential for the binary reshaping pipeline
Captum	XAI library for Grad-CAM	Purpose-built PyTorch

	heatmap generation integrated with PyTorch models	explainability library with GradCam implementation; seamless integration with PyTorch model hooks
CUDA	GPU acceleration for deep learning model training and inference	NVIDIA's parallel computing platform enabling practical CNN training on large malware image datasets; reduces training time from hours to minutes
SQLite	Local relational database for detection event logging and historical trend analysis	Zero-configuration embedded database; ideal for local event logging without server infrastructure; Python sqlite3 module included in standard library

1.10 Project Contribution

MalTwin introduces a set of original technical and conceptual contributions that collectively advance the state of IIoT malware detection beyond existing partial solutions:

- **Safe IIoT Malware Simulation via Fully Containerized Digital Twin:** MalTwin is among the first research frameworks to combine a fully software-based Digital Twin IIoT simulation environment (Docker + Mininet) with AI-driven malware classification in a unified pipeline. The Digital Twin enables safe malware execution and behavioral observation with industrial protocol fidelity (Modbus TCP, MQTT) without any risk to physical equipment, directly addressing the critical research barrier of IIoT hardware unavailability.
- **End-to-End IIoT-Focused Binary-to-Image Detection Pipeline:** Unlike prior work that applied visualization techniques to general Windows PE malware, MalTwin extends the binary-to-image approach to IIoT-relevant executable formats (ELF binaries from Linux-based IIoT devices) and integrates the full pipeline from raw binary to classification result within a single deployable framework.
- **Domain-Adapted Dataset Enhancement for IIoT Malware:** MalTwin applies augmentation and balancing strategies specifically calibrated for the grayscale binary image domain, addressing the dual challenges of dataset scarcity and class imbalance that are particularly severe for IIoT-specific malware samples compared to general malware corpora.
- **Integrated Grad-CAM Explainability for Binary Image Classification:** MalTwin uniquely integrates Grad-CAM-based explainability directly into the malware detection pipeline, generating attribution heatmaps overlaid on grayscale binary images. This enables security analysts to understand which specific byte regions of an executable drive classification decisions, a capability absent from all identified related systems.
- **Automated MITRE ATT&CK for ICS Threat Intelligence Mapping:** The automated

reporting module maps each detection output to the MITRE ATT&CK for Industrial Control Systems framework, translating AI classification results into the standardized adversary behavior language used by industrial security operations centers.

- **Unified Modular Research Architecture with Full Reproducibility:** The eight-module architecture is designed to allow independent substitution or extension of any component without redesigning the surrounding pipeline. All implementation uses public datasets, standard Python libraries, and documented configuration, ensuring full experimental reproducibility.

These contributions collectively position MalTwin as a more comprehensive research framework than any single identified prior system, which addresses at most two of these six dimensions simultaneously.

1.11 Relevance to Course Modules

MalTwin serves as a comprehensive synthesis of core academic principles by aligning directly with multiple domains of your coursework through the following applications:

- **Cybersecurity and Malware Analysis:** The framework applies threat modeling, malware classification taxonomy, and MITRE ATT&CK for ICS mapping to address real-world IIoT attack vectors.
- **Artificial Intelligence and Machine Learning:** This domain is put into practice through the intelligent detection module, which utilizes supervised deep learning and Convolutional Neural Networks for multi-class image classification while exercising hyperparameter tuning and overfitting mitigation.
- **Computer Vision and Image Processing:** The project draws heavily from this coursework through the binary-to-image conversion process, pixel normalization, and the generation of Grad-CAM heatmaps for spatial feature extraction.
- **Network Security and Industrial Protocols:** These concepts are represented by the Digital Twin module, which emulates Modbus TCP and MQTT communication protocols within a realistic network topology to analyze industrial security.
- **Software Engineering and Project Management:** Principles from these courses are integrated through modular system architecture, WBS task decomposition, and the application of SDLC methodologies for requirement analysis and documentation.
- **Database Systems:** Academic knowledge is applied via SQLite-based event logging and relational schema design for historical trend analysis.
- **Operating Systems and Virtualization:** The project utilizes these concepts by leveraging Docker containerization and Linux namespaces to manage process isolation and virtual networking within the simulation environment.

2 Chapter 2: Requirement Analysis

This chapter presents the complete requirement analysis for the MalTwin framework. It begins by identifying user classes and their characteristics, defines the operating environment and implementation constraints, and presents a complete Use Case Diagram showing system actors and their interactions. Functional requirements are systematically derived using Mockup-Based Requirement Analysis for user-facing features and Event-Response Tables for backend processes. The chapter concludes with comprehensive non-functional requirements expressed in measurable, verifiable terms, and detailed external interface specifications.

2.1 User Classes and Characteristics

Four distinct user classes have been identified for the MalTwin framework. Each class differs in technical background, interaction frequency, primary goals, and the subset of system features they engage with.

Table 2.1: MalTwin User Classes and Characteristics

User Class	Description and Characteristics
Security Researcher	A graduate-level cybersecurity researcher or academic project team member who uses MalTwin to conduct experiments in IIoT malware detection. Primary activities include configuring and running the Digital Twin simulation, training and evaluating detection models, tuning augmentation parameters, and generating research-grade evaluation reports. This user interacts with all eight system modules. Frequency of use is typically daily during active research phases. There are an estimated 2 primary users within the project team, with potential for broader academic adoption.
Security Analyst	An industrial cybersecurity practitioner or SOC analyst who uses the MalTwin Streamlit dashboard to analyze suspicious binary files from IIoT environments. Primary activities include uploading binary samples, inspecting grayscale visualizations, reviewing detection predictions and confidence scores, viewing XAI heatmap explanations, and downloading forensic reports. This user interacts primarily through the Streamlit dashboard GUI. Frequency of use is as-needed during incident investigation or malware triage. There are an estimated 3–10 users in a typical deployment.
System Administrator	An IT or OT infrastructure administrator responsible for deploying, maintaining, and monitoring the MalTwin framework. Primary activities include installing dependencies, managing Docker containers, configuring the Streamlit server, monitoring system resource usage, performing software updates, and backing up the detection event database. This user does not directly interact with malware analysis features under normal operations. Frequency of use involves periodic maintenance tasks and as needed for incident response.
Academic Supervisor / Evaluator	A faculty member, academic supervisor, or FYP committee evaluator who reviews MalTwin's outputs for assessment purposes. Primary activities include reviewing model performance evaluation reports, examining forensic output quality, assessing the completeness of XAI explanations, and evaluating the quality and documentation of the framework. This user primarily consumes system outputs through the dashboard and generated documentation rather than operating the system directly. Frequency of use involves periodic review during project milestones and final evaluation.

2.2 Operating Environment

The following operating environment requirements define the hardware, software, network, and platform conditions under which the MalTwin framework must function correctly.

OE-1: The MalTwin system shall operate on Linux-based host machines running Ubuntu 22.04 LTS or a compatible Debian-based distribution, with kernel version 5.15 or later supporting Docker namespace isolation and eBPF-based network monitoring.

OE-2: The Digital Twin simulation module shall operate within Docker Engine version 29.x or later, with Docker Compose support for multi-container orchestration and Mininet 2.3.0 for software-defined IIoT network topology emulation.

OE-3: The Streamlit-based dashboard shall be accessible via modern web browsers including Google Chrome (version 120+), Mozilla Firefox (version 120+), and Microsoft Edge (version 120+), connecting over localhost (127.0.0.1) or local area network HTTP.

OE-4: GPU-accelerated training shall be supported using CUDA 13.1 on NVIDIA GPUs with a minimum of 6 GB VRAM. CPU-only operation shall remain fully supported for inference, evaluation, and dashboard use cases without GPU hardware.

OE-5: The system shall operate in a network-isolated environment during malware analysis; all Docker containers used for the Digital Twin shall be assigned to a dedicated Docker bridge network not connected to external or host network interfaces.

OE-6: The framework shall operate correctly on host machines with a minimum hardware specification of 8-core CPU, 16 GB RAM, and 100 GB available local disk storage. Training workloads may require up to 32 GB RAM depending on dataset size.

OE-7: Python 3.14.x shall be the required interpreter version; the framework shall not depend on system-level Python packages and shall operate within a dedicated virtual environment or Conda environment to avoid dependency conflicts.

OE-8: The framework shall support offline operation without internet connectivity for all core analysis, detection, and reporting functions; MITRE ATT&CK mapping shall use a locally stored reference JSON file.

2.3 Design and Implementation Constraints

The following constraints impose fixed requirements on the design and implementation choices available to the development team.

CON-1: The entire MalTwin framework shall be implemented in Python 3.14.x to ensure a unified language ecosystem across all modules and full compatibility with the selected scientific computing, machine learning, and visualization libraries.

CON-2: All IIoT simulation and malware binary execution activities shall be performed exclusively within Docker containers using Mininet-based virtual network namespaces, ensuring complete isolation from the host operating system and all physical network interfaces.

CON-3: The malware detection model shall be implemented using PyTorch 2.9.x as the primary deep learning framework. TensorFlow/Keras 2.20.x may be used for comparative experimentation but shall not be the primary deployment framework.

CON-4: The user-facing dashboard interface shall be built exclusively using Streamlit 1.52.x. No alternative frontend framework (React, Flask, Django) shall be used for the primary analyst interface.

CON-5: Under no circumstances shall any malware binary file be executed, stored, or processed outside of the containerized Docker simulation environment on the host file system.

CON-6: All datasets used for model training and evaluation shall be sourced exclusively from publicly available, legally accessible repositories (Malimg, EMBER, VirusShare with registered access, IoT-23) to ensure research reproducibility and legal compliance.

CON-7: Forensic reports shall be generated locally using the FPDF library in PDF format and Python's built-in json module for JSON format. No external cloud-based document generation, storage, or transmission services shall be used.

CON-8: The local detection event database shall use SQLite via Python's built-in sqlite3 module. No external database server (PostgreSQL, MySQL) shall be required for baseline operation.

CON-9: All SHA-256 hash computations for uploaded binary files shall be performed locally using Python's hashlib standard library module. No file content or hash value shall be transmitted to any external online hash lookup or reputation service.

2.4 Use Case Diagram

The Use Case Diagram below identifies the four primary actors of the MalTwin framework, Security Researcher, Security Analyst, System Administrator, and Academic Evaluator, and maps their interactions with the fourteen core use cases within the MalTwin system boundary.

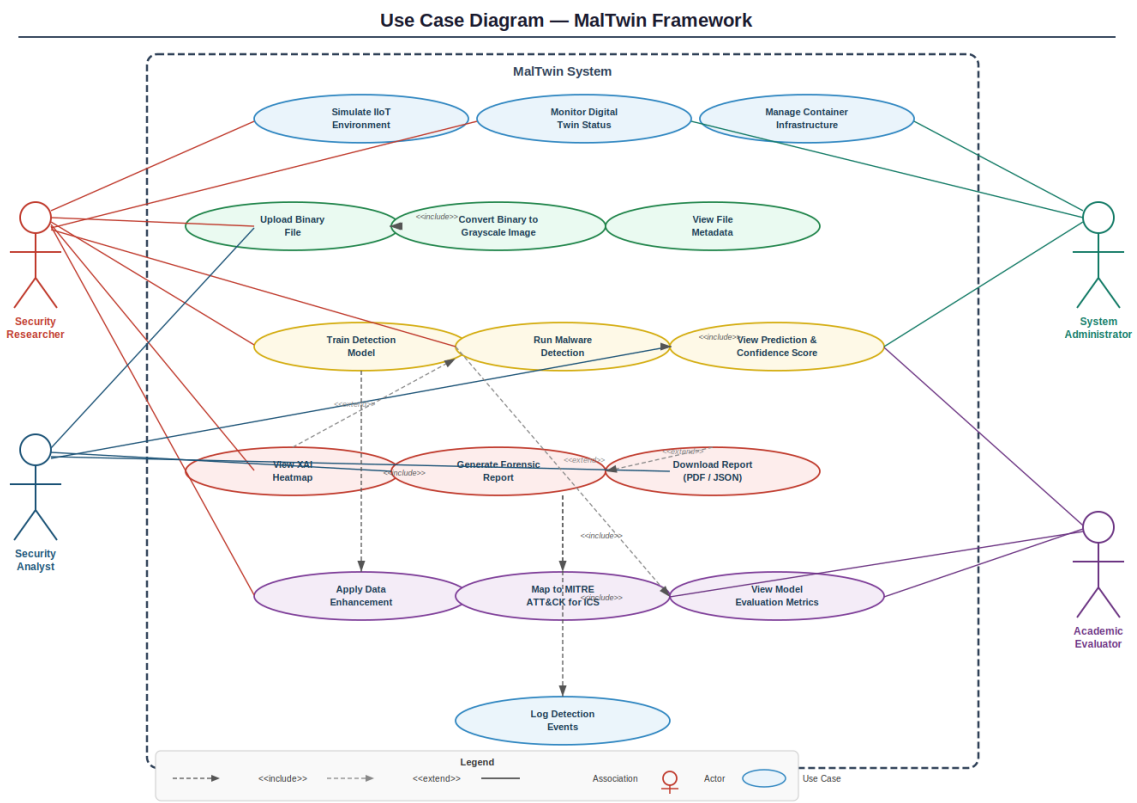

Figure 2.1: Use Case Diagram, MalTwin Framework

The Security Researcher is the most privileged actor and has associations with the broadest set of use cases: Simulate IIoT Environment, Monitor Digital Twin Status, Upload Binary File, Convert Binary to Grayscale Image, Train Detection Model, Apply Data Enhancement, Run Malware Detection, and View XAI Heatmap. The Security Analyst is the primary dashboard user, interacting with Upload Binary File, Run Malware Detection, View Prediction and Confidence Score, View XAI Heatmap, Generate Forensic Report, and Download Report. The System Administrator's use cases are infrastructure-focused: Manage Container Infrastructure, Simulate IIoT Environment, and Monitor Digital Twin Status. The Academic Evaluator's use cases are output-consumption oriented: View Model Evaluation Metrics, View Prediction and Confidence Score, and Download Report.

Table 2.2: Use Case Summary

Use Case Name	Primary Actor(s)	Relationship Type
Simulate IIoT Environment	Security Researcher, System Administrator	Association
Monitor Digital Twin Status	Security Researcher, System Administrator	Association
Manage Container Infrastructure	System Administrator	Association
Upload Binary File	Security Researcher, Security Analyst	Association
Convert Binary to Grayscale Image	System (automated)	<<include>> from Upload Binary File
View File Metadata	Security Researcher, Security Analyst	Association
Train Detection Model	Security Researcher	Association
Apply Data Enhancement	System (automated)	<<include>> from Train Detection Model
Run Malware Detection	Security Researcher, Security Analyst	Association
View Prediction & Confidence Score	Security Researcher, Security Analyst, Academic Evaluator	Association
View XAI Heatmap	Security Researcher, Security Analyst	<<extend>> from Run Malware Detection
Generate Forensic Report	Security Analyst, Security Researcher	Association
Map to MITRE ATT&CK for ICS	System (automated)	<<include>> from Generate Forensic Report
Log Detection Events	System (automated)	<<include>> from Generate Forensic Report
Download Report (PDF/JSON)	Security Analyst, Academic Evaluator	<<extend>> from Generate Forensic Report
View Model Evaluation Metrics	Security Researcher, Academic Evaluator	<<extend>> from Train Detection Model

2.4.1 Detailed Use Case Descriptions

The following table provides structured descriptions of the five most critical use cases in the MalTwin framework, including actor participation, preconditions, main flow, alternate flows, and postconditions.

Table 2.3: Detailed Use Case Descriptions, Selected MalTwin Use Cases

Field	UC-01: Upload Binary File and Convert to Grayscale Image
Actor(s)	Security Researcher (primary), Security Analyst (secondary)
Preconditions	1. The MalTwin dashboard is running and accessible via browser. 2. The user has a PE (.exe, .dll) or ELF binary file available for upload. 3. File size does not exceed 50 MB.
Main Success Flow	1. User navigates to the Binary Upload and Visualization screen via the sidebar menu. 2. User drags and drops or selects the binary file using the file picker control. 3. System validates file format (PE/ELF header check) and file size. 4. System reads the binary as a raw byte stream and reshapes it into a 2D numpy array. 5. System renders the array as a 128×128 grayscale PNG image and displays it in the main panel. 6. System computes the SHA-256 hash of the file and displays file metadata (name, size, format, hash). 7. System displays the pixel intensity histogram of the generated grayscale image.
Alternate Flows	A1, Invalid Format: File fails PE/ELF header validation. System displays error: 'Unsupported file format. Please upload a valid PE or ELF binary.' Upload is rejected; no image is generated. A2, File Too Large: File exceeds 50 MB. System displays error: 'File exceeds maximum size limit of 50 MB.' Upload is rejected before reading begins.
Postconditions	Grayscale image is stored in session memory. SHA-256 hash and metadata are available for downstream detection and reporting. Detection control becomes active.

Field	UC-02: Run Malware Detection and View Prediction
Actor(s)	Security Researcher (primary), Security Analyst (primary)
Preconditions	1. A grayscale binary image has been generated in the current session (UC-01 complete). 2. A trained detection model is loaded in the system. 3. The user is on the Malware Detection screen.
Main Success Flow	1. User activates the 'Run Detection' control on the Malware Detection screen. 2. System loads the serialized model weights and prepares the grayscale image (normalize, add batch dimension). 3. System executes a forward pass through the CNN model. 4. System extracts the top-1 predicted malware family label and its associated confidence score.

	5. System displays the predicted label, color-coded confidence bar, and full per-class probability chart. 6. System retrieves and displays the MITRE ATT&CK for ICS mapping for the predicted malware family. 7. Detection event is automatically logged to the SQLite database.
Alternate Flows	A1, Model Not Loaded: System displays 'No trained model is available. Please train or load a model before running detection.' A2, Low Confidence Result (<50%): System displays Amber warning advisory alongside the prediction: 'Low confidence detection. Results may be unreliable, manual verification recommended.' A3, Database Write Failure: System retries log write once; if retry fails, displays non-blocking advisory while still showing detection result.
Postconditions	Detection result (label, confidence, MITRE mapping) is available in the current session. Detection event is logged to the local database. Download Report control becomes active.

Field	UC-03: Generate and Download Forensic Report
Actor(s)	Security Analyst (primary), Security Researcher (secondary)
Preconditions	1. A detection result is available in the current session (UC-02 complete). 2. User is on the Malware Detection screen.
Main Success Flow	1. User selects desired report format (PDF or JSON) using the format selector control. 2. User activates the 'Download Report' control. 3. System retrieves detection result data from session memory. 4. System queries the local MITRE ATT&CK for ICS JSON database for the predicted family. 5. System compiles report content: file name, SHA-256, format, timestamp, predicted label, confidence, MITRE mapping, XAI heatmap (if generated). 6. System generates the PDF using FPDF or JSON using Python json.dumps(). 7. System serves the generated file as a browser download.
Alternate Flows	A1, No MITRE Mapping: Report includes section: 'MITRE ATT&CK mapping not available for predicted family.' Report generation continues normally. A2, PDF Generation Failure: System attempts JSON fallback and notifies user: 'PDF generation failed. JSON report downloaded instead.'
Postconditions	Forensic report file is downloaded to the user's browser download directory. Report generation event is logged with timestamp.

Field	UC-04: Simulate IIoT Environment via Digital Twin
Actor(s)	Security Researcher (primary), System Administrator (secondary)
Preconditions	1. Docker Engine is running on the host. 2. All required Docker container images are available locally. 3. Ports 502 (Modbus) and 1883 (MQTT) are not occupied by other processes.

Main Success Flow	1. User navigates to the Digital Twin Monitor screen. 2. User configures simulation parameters: number of nodes, traffic volume, malware sample to deploy (optional). 3. User activates the 'Start Simulation' control. 4. System verifies Docker availability and port availability. 5. System initializes Docker containers and Mininet topology with the configured node structure. 6. System starts Modbus and MQTT traffic generators within the simulation network. 7. System begins streaming live traffic log and node status updates to the dashboard. 8. If malware sample selected: system deploys sample within isolated container and monitors behavioral artifacts.
Alternate Flows	A1, Docker Unavailable: System displays 'Docker Engine is not running. Please start Docker before initiating simulation.' A2, Port Conflict: System displays 'Port 502 or 1883 is in use. Please free the required ports and retry.' A3, Container Timeout: If containers fail to initialize within 30 seconds, simulation is aborted with error notification.
Postconditions	IIoT simulation is running; live traffic log and node status are streaming to dashboard. Traffic PCAP capture is ongoing for potential export.

Field	UC-05: Train Detection Model
Actor(s)	Security Researcher (primary)
Preconditions	1. Malware dataset has been downloaded and organized in the Malimg directory structure. 2. Python environment with PyTorch is correctly configured. 3. Data Enhancement module has been run (dataset is balanced and augmented).
Main Success Flow	1. Researcher configures training hyperparameters: model architecture, learning rate, batch size, number of epochs, train/validation split ratio. 2. Researcher activates the training pipeline via command-line or dashboard training screen. 3. System applies data enhancement and balancing to the training split. 4. System instantiates the CNN model with the configured architecture. 5. System trains the model for the configured number of epochs, logging per-epoch loss and accuracy. 6. System evaluates the trained model on the validation set after each epoch; saves best weights based on validation accuracy. 7. After training completes, system evaluates the final model on the held-out test set and displays full metrics: accuracy, precision, recall, F1-score, confusion matrix.
Alternate Flows	A1, Out of Memory: System catches CUDA out-of-memory error and displays: 'GPU memory insufficient. Reduce batch size and retry.' A2, Dataset Not Found: System displays: 'Dataset directory not found. Please configure the dataset path in settings.'

Postconditions	Best-performing model weights are saved to disk in .pt format. Evaluation metrics are logged and accessible in the Model Evaluation Metrics dashboard screen.

2.5 Requirement Identifying Technique

For MalTwin, two complementary techniques are used to identify and document system requirements:

1. **Use Case-Driven Requirement Analysis**, to model user interactions with the system including file uploads, Digital Twin configuration, dataset management, model training, and dashboard-based result visualization.
2. **Module-Based Event-Response Analysis**, to represent automated backend processes such as binary-to-image conversion, data enhancement, model training, XAI explanation generation, and automated threat reporting.

2.5.1 Mockup-Based Requirement Analysis

The Mockup-Based Requirement Analysis technique captures how users observe and interpret system outputs through visualization-oriented user interfaces. In MalTwin the UI is **strictly used for monitoring and result visualization**, and does not provide control over data processing, model configuration, or execution.

Each mockup screen is labeled M1–M4, and UI elements are mapped to corresponding Functional Requirements (FR). This ensures that system outputs, processing states, and evaluation results are clearly communicated to users in a structured and interpretable manner without exposing backend complexity or allowing modification of system behavior.

2.6 Functional Requirements

Functional requirements are organized by mockup screen origin. Each requirement is uniquely identified (FR-ID), expressed as a testable 'the system shall' statement, referenced to its originating mockup screen, and accompanied by applicable business rules where domain-specific constraints govern the behavior.

2.6.1 Mockup M1, Main Dashboard Screen

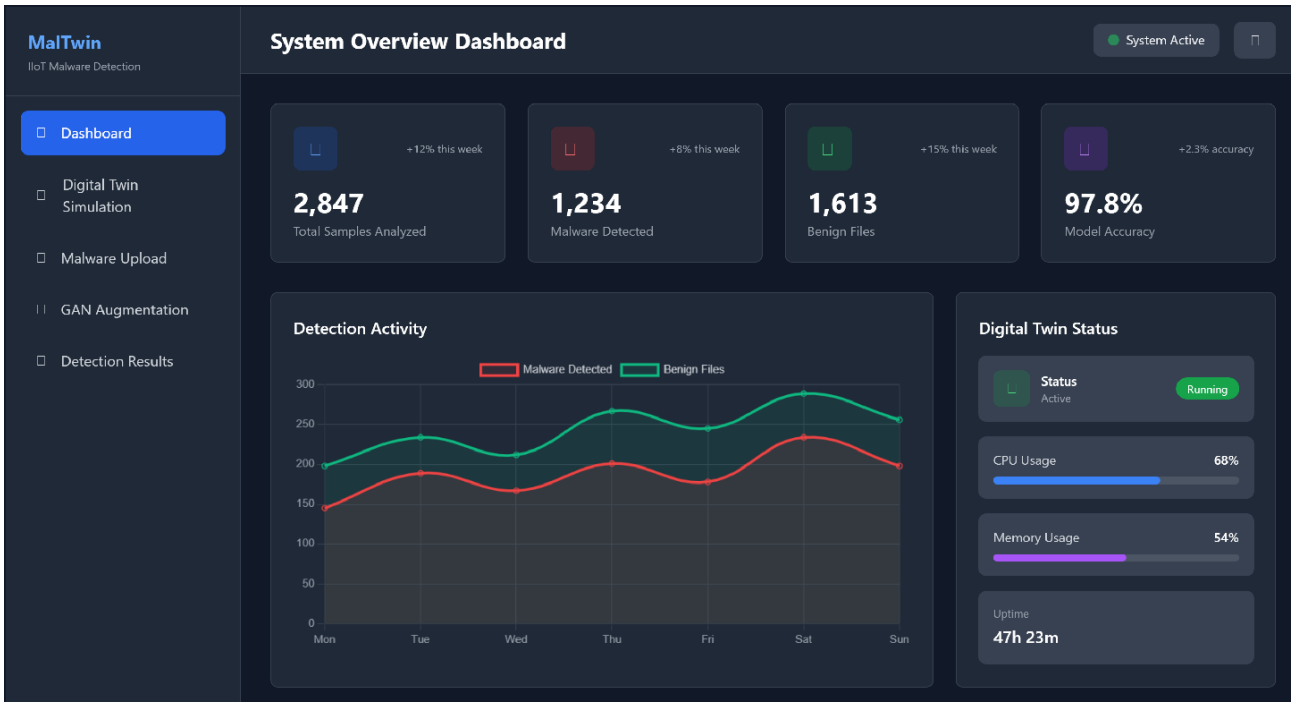

Table 2.3: Functional Requirements Derived from Mockup M1, Main Dashboard

Feature (from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
System Status Overview	FR1.1: The system shall display the current operational status (Active / Inactive / Error) of all eight MalTwin modules on the main dashboard landing screen.	Status indicators shall refresh automatically every 5 seconds without requiring a manual page reload.
Detection Statistics Panel	FR1.2: The system shall display cumulative detection statistics including total files analyzed, total malware detected, and total benign files confirmed in real-time.	Statistics shall be computed from the local SQLite detection event log; they shall persist across dashboard sessions.
Sidebar Navigation Menu	FR1.3: The system shall provide a persistent sidebar navigation menu enabling users to switch between all dashboard screens without page reload.	Navigation menu items shall reflect only currently available and initialized modules; unavailable modules shall be grayed out.
Recent Detection Feed	FR1.4: The system shall display a scrollable feed of the five most recent detection events on the main dashboard, showing file name, predicted label, confidence score, and timestamp.	Feed shall update in real-time as new detections are logged; oldest entries shall be automatically removed when the list exceeds five items.

2.6.2 Mockup M2, Digital Twin Monitor Screen

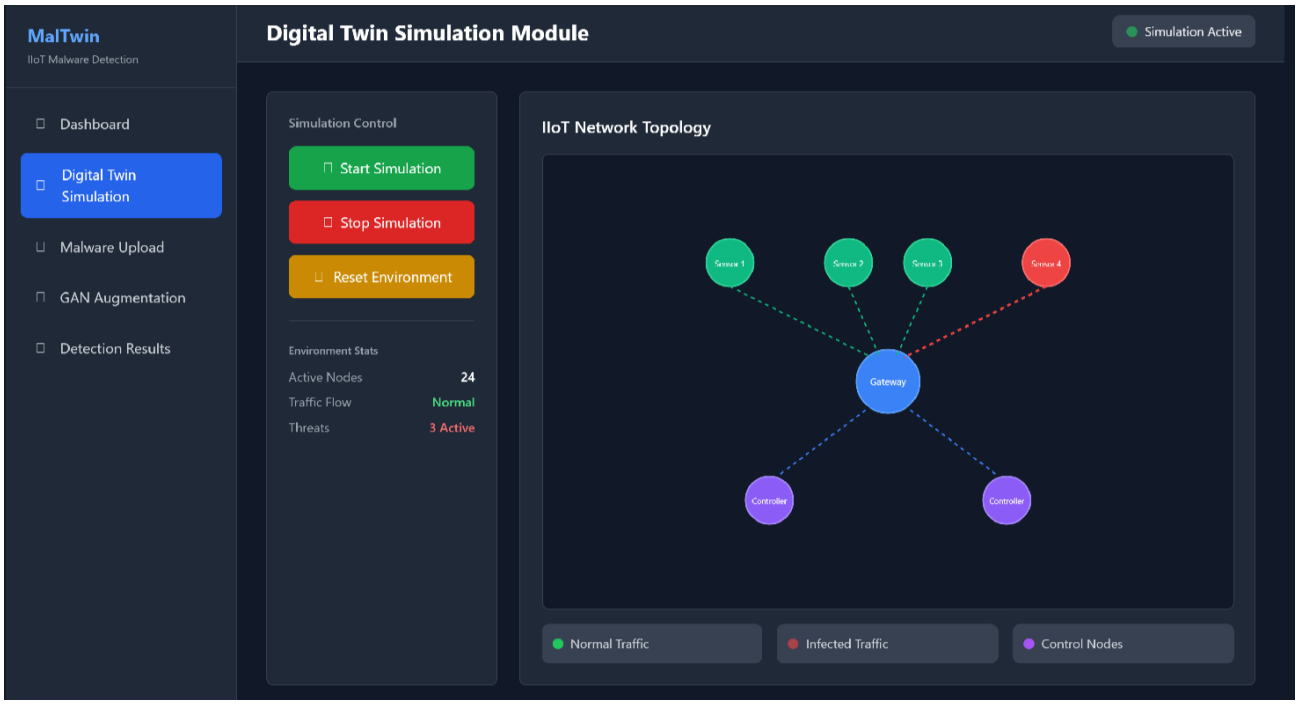

Table 2.4: Functional Requirements Derived from Mockup M2, Digital Twin Monitor

Feature (from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
Start / Stop Simulation	FR2.1: The system shall allow the user to start and stop the Digital Twin IIoT simulation environment through dedicated on-screen controls.	Simulation start shall only be permitted if Docker Engine is running and all required container images are present locally.
Live Traffic Log	FR2.2: The system shall display a scrollable, real-time log of network traffic events captured within the Digital Twin simulation, including timestamp, source node, destination node, and protocol.	Log entries shall be retained for the duration of the current simulation session only; logs shall clear on simulation restart.
Node Status Panel	FR2.3: The system shall display the current status of each emulated IIoT node (Active, Inactive, Infected) in a visual panel, updated in real-time.	Node status shall reflect the most recent state reported by the containerized Mininet topology; infected status shall trigger a visible alert indicator.
Protocol Traffic Distribution	FR2.4: The system shall display a real-time pie chart showing the distribution of captured traffic by protocol type (Modbus, MQTT, Other).	Chart shall update at 5-second intervals during an active simulation session.

2.6.3 Mockup M3, Binary Upload and Visualization Screen

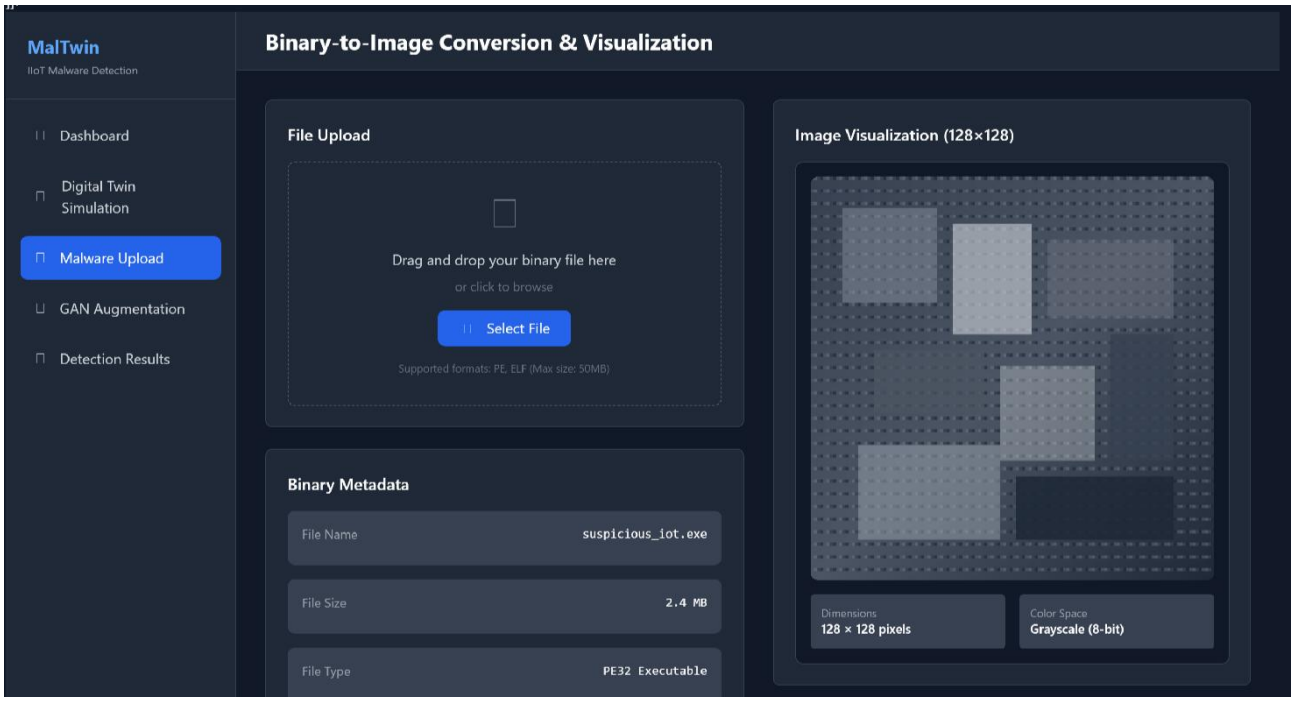

Table 2.5: Functional Requirements Derived from Mockup M3, Binary Upload and Visualization

Feature (from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
Binary File Upload Control	FR3.1: The system shall allow the user to upload a binary executable file for analysis through a drag-and-drop or file-picker control.	Accepted formats: PE (.exe, .dll) and ELF binaries. Maximum file size: 50 MB. Files exceeding size limit shall be rejected with a descriptive error message.
Grayscale Image Display	FR3.2: The system shall convert the uploaded binary to a grayscale image and display it in the main panel within 3 seconds of upload completion.	Output image shall be standardized to 128×128 pixels via bilinear interpolation prior to display.
File Metadata Display	FR3.3: The system shall display the following file metadata after upload: file name, file size in bytes, file format type (PE/ELF), and SHA-256 hash.	SHA-256 hash shall be computed locally; no external hash lookup services shall be queried.
Pixel Intensity Histogram	FR3.4: The system shall display a pixel intensity histogram of the converted grayscale image showing the distribution of byte values across the binary.	Histogram shall use 256 bins (one per byte value 0–255) and update immediately when a new image is generated.

2.6.4 Mockup M5, Malware Detection and Prediction Screen

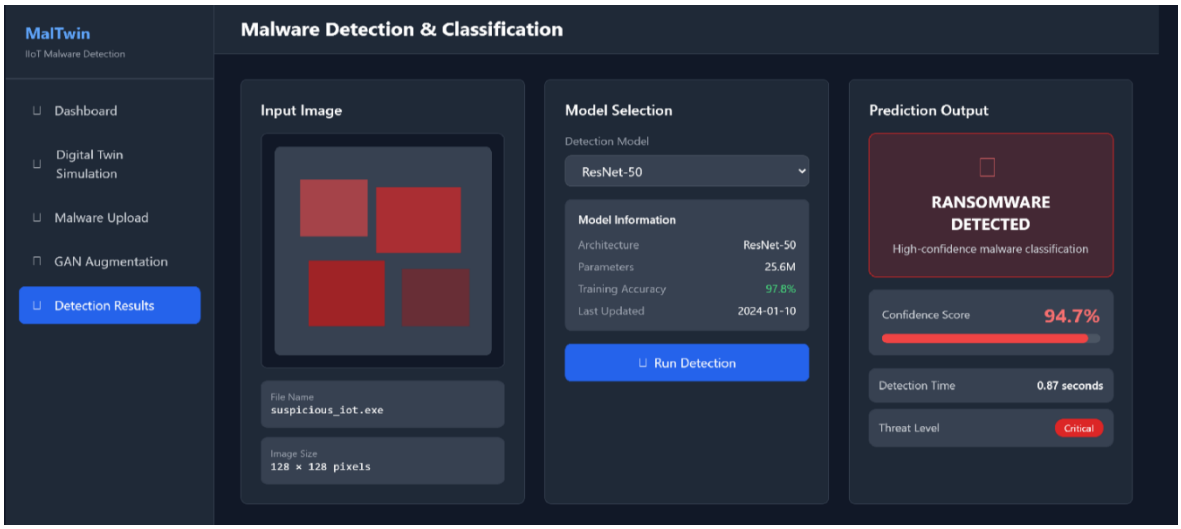

Table 2.6: Functional Requirements Derived from Mockup M5, Malware Detection

Feature (from UI)	Functional Requirement (FR-ID: Statement)	Business Rule
Run Detection Button	FR5.1: The system shall run the trained malware detection model on the uploaded grayscale image and return a classification result when the user activates the detection control.	Detection shall only execute if a trained model is loaded and a converted grayscale image is available; otherwise the control shall be disabled.
Prediction Label and Confidence Bar	FR5.2: The system shall display the top-1 predicted malware family label and a horizontal confidence bar showing the prediction confidence as a percentage.	Confidence bar shall use color coding: Green (≥80% confidence), Amber (50–79%), Red (<50%). Low-confidence predictions shall display a warning advisory to the analyst.
Per-Class Probability Chart	FR5.3: The system shall display a horizontal bar chart showing the model's predicted probability for all malware family classes, sorted in descending order.	Chart shall display all classes from the training dataset, with zero-probability classes shown as empty bars rather than omitted.
XAI Heatmap Toggle	FR5.4: The system shall provide an optional control allowing the user to request Grad-CAM XAI heatmap generation; the heatmap shall be displayed overlaid on the grayscale image when requested.	Heatmap generation is optional and on-demand; it shall not execute automatically during detection to preserve performance.
MITRE ATT&CK Mapping Display	FR5.5: The system shall display the MITRE ATT&CK for ICS tactics and techniques associated with the predicted malware family beneath the detection result.	If no MITRE mapping exists for the predicted family in the local reference database, the system shall display 'MITRE mapping

		not available for this family' rather than an error.
Report Download Control	FR5.6: The system shall allow the user to download a forensic report in PDF or JSON format after a detection result is available, through clearly labeled download buttons.	Report shall include: file name, SHA-256 hash, predicted family, confidence score, timestamp, XAI heatmap (if generated), and MITRE ATT&CK mapping.

2.6.5 Event-Response Table, Backend Process Requirements

The following Event-Response Table captures functional requirements for MalTwin's backend processes that are not directly triggered by user interface interactions.

Table 2.7: Event-Response Table, MalTwin Backend Processes

Event Name	Trigger Source	System Response	Exception Handling
Binary File Uploaded	User via Dashboard Upload Control	FR-B1: Parse binary header to identify PE/ELF format. Read raw byte stream. Reshape to 2D array. Render 128×128 grayscale PNG. Compute SHA-256 hash. Store image in session memory.	Reject file if format validation fails or file exceeds 50 MB. Display descriptive error message. Clear previous session image.
Detection Run Requested	User via Detection Control	FR-B2: Load serialized model weights. Preprocess grayscale image (normalize, add batch dimension). Execute forward pass. Extract top-1 label and confidence. Return class probability distribution.	Return error notification if model weights file is missing or image preprocessing fails. Log error with timestamp.
Detection Result Available	Detection Module (internal)	FR-B3: Write detection event record to SQLite database including timestamp, SHA-256 hash, predicted label, confidence score, and file name.	If database write fails, retry once after 2 seconds. If retry fails, display non-blocking advisory; do not block result display.
Forensic Report Requested	User via Download Control	FR-B4: Query local MITRE ATT&CK for ICS JSON database for the predicted malware family. Compile report content. Generate PDF using FPDF and JSON	If MITRE mapping is absent for predicted family, include 'No mapping available' note in report. Do not abort report generation.

		using built-in json module. Serve files for browser download.	
Digital Twin Start Requested	User via Digital Twin Control	FR-B5: Verify Docker Engine availability. Pull or validate required container images. Initialize Mininet topology with configured node count. Start Modbus and MQTT traffic generators. Begin log streaming.	Abort with error notification if Docker daemon is not running, port conflicts exist (502, 1883), or container initialization fails within 30 seconds.
Grad-CAM Heatmap Requested	User via XAI Control	FR-B6: Register Grad-CAM hooks on the final convolutional layer of the loaded model. Execute targeted backward pass. Compute gradient-weighted activation map. Resize heatmap to 128×128. Overlay on grayscale input image using jet colormap. Display result.	If Grad-CAM computation fails (e.g., model architecture incompatible), display error message and advise user to verify model compatibility.

2.7 Non-Functional Requirements

Non-functional requirements define the quality attributes of the MalTwin framework. All requirements are expressed in specific, quantitative, and verifiable terms to enable objective testing and validation. These requirements supplement the functional specifications and define the acceptable quality envelope for the system.

2.7.1 Reliability

REL-1: The MalTwin detection module shall produce identical classification results for the same binary input file across repeated inferences on the same hardware configuration, with a confidence score variance of less than 0.5% across 10 consecutive runs (deterministic inference mode, no dropout at test time).

REL-2: The system shall handle corrupt, truncated, or malformed binary files without crashing. All file parsing exceptions shall be caught, logged with a full stack trace to the system log, and communicated to the user with a descriptive error message within 3 seconds of upload completion.

REL-3: The Digital Twin simulation environment shall achieve a Mean Time Between Failure (MTBF) of at least 4 hours of continuous operation before requiring a manual container restart under normal simulated traffic loads.

REL-4: The detection event SQLite database shall maintain data integrity across system restarts; no detection records shall be lost due to unclean shutdown. Write-ahead logging (WAL) mode shall be enabled for the SQLite connection.

REL-5: The Streamlit dashboard shall handle concurrent file uploads from at least 2 simultaneous user sessions without data collision or cross-session contamination of analysis results.

2.7.2 Usability

USE-1: A security analyst with no prior MalTwin experience but familiarity with cybersecurity tools shall be able to successfully upload a binary file, run malware detection, and view the prediction result within 5 minutes of first accessing the dashboard, without consulting the user documentation.

USE-2: The dashboard shall display all detection results, confidence scores, and XAI heatmaps using visual indicators (color-coded bars, overlaid heatmaps with labeled colormaps) interpretable without additional explanation or technical background in machine learning.

USE-3: All user-facing error messages shall include: (a) a plain-language description of what went wrong, (b) the likely cause of the error, and (c) a suggested corrective action. No raw Python exception tracebacks shall be displayed to end users.

USE-4: The system shall provide inline tooltips or help text for all technical terms displayed in the dashboard including 'confidence score', 'Grad-CAM heatmap', 'MITRE ATT&CK tactic', and 'malware family'.

USE-5: The forensic report download shall complete within 10 seconds and shall produce a file that opens correctly in standard PDF viewers (Adobe Acrobat, browser PDF viewer) and JSON editors without post-processing.

2.7.3 Performance

PER-1: Binary-to-image conversion for a file of up to 10 MB shall complete within 3 seconds on the reference hardware specification (8-core CPU, 16 GB RAM, Python 3.14). Files up to 50 MB shall complete conversion within 10 seconds.

PER-2: Model inference for a single 128×128 grayscale image shall complete within 5 seconds on CPU-only hardware and within 1 second when CUDA GPU acceleration is available on the reference GPU specification (NVIDIA GPU, 6 GB+ VRAM).

PER-3: The Streamlit dashboard shall fully render all UI components and become interactive within 4 seconds of initial page load on a local network connection.

PER-4: Grad-CAM heatmap generation shall complete within 8 seconds of user request on CPU hardware and within 3 seconds with GPU acceleration.

PER-5: Forensic report generation in both PDF and JSON formats shall complete within 10 seconds of the detection result becoming available, including MITRE ATT&CK lookup and FPDF document compilation.

PER-6: The detection event SQLite database shall support insertion of at least 100,000 detection records while maintaining query response times of under 500 milliseconds for the five most recent record retrieval query used by the dashboard feed.

2.7.4 Security

SEC-1: All uploaded binary files shall be processed exclusively within Docker container boundaries. No uploaded file content shall be written to the host file system outside of designated, write-protected temporary directories with automatic cleanup on session end.

SEC-2: The Digital Twin simulation network shall be assigned to a dedicated Docker bridge network (malware-sim-net) that is explicitly not connected to any host network interfaces or external networks. Network namespace isolation shall be verified at simulation startup.

SEC-3: The detection event SQLite database file shall be stored with file permissions 600 (owner read/write only) on the host file system, preventing unauthorized read or modification by other system users.

SEC-4: SHA-256 hash computation shall be performed entirely locally using Python's hashlib standard library. No binary file content, file hash, or analysis result shall be transmitted to any external service, API, or cloud platform.

SEC-5: The Streamlit dashboard shall be bound to localhost (127.0.0.1) by default, preventing external network access. Administrators who choose to expose the dashboard on a local area network interface shall be explicitly warned of the associated security implications during startup.

2.8 External Interface Requirements

2.8.1 User Interface Requirements

The MalTwin user interface shall be implemented as a Streamlit multi-page web application. The interface shall adopt a consistent dark-themed professional color scheme appropriate for cybersecurity analysis contexts, with primary colors drawn from a navy-blue and white palette with accent colors used exclusively for status indicators. All screens shall follow a two-column layout (sidebar navigation left, main content right) as the default structure. Font sizes shall be consistent: section headers at 18pt, body text at 13pt, and table cells at 11pt minimum for readability.

File upload controls shall clearly specify the accepted file formats (.exe, .dll, ELF binaries) and the maximum file size limit (50 MB) within the control label or adjacent help text. All interactive buttons shall include descriptive labels and shall visually indicate loading state (spinner animation) during asynchronous operations such as model inference and heatmap generation. Image display panels shall always include a caption identifying the image content (e.g., 'Grayscale visualization of uploaded binary, 128×128 pixels'). All charts and graphs shall include axis labels, units, and titles. Color-coded indicators shall always be accompanied by text labels to ensure accessibility for color-blind users.

2.8.2 Software Interfaces

SI-1: Docker Engine API, MalTwin shall interface with the local Docker Engine via the Docker Python SDK to create, start, stop, monitor, and remove containers for the Digital Twin simulation. Container health status shall be polled at 5-second intervals.

SI-2: Mininet Python API, The Digital Twin module shall use Mininet's Python API (mn module) to programmatically define and instantiate IIoT network topologies within Docker-networked namespaces, including node creation, link configuration, and protocol traffic generation.

SI-3: PyTorch Model Serialization API, The detection module shall load trained model weights from .pt files using torch.load() with explicit device mapping. Models shall be saved and loaded using PyTorch's state_dict serialization format for portability.

SI-4: Captum Attribution API, The XAI module shall interface with Captum's GradCam class, registering forward and backward hooks on the target convolutional layer of the loaded PyTorch model to compute class-discriminative gradient activations.

SI-5: Malimg Dataset File System Interface, The dataset module shall read malware image samples from the Malimg dataset directory structure (one subdirectory per malware family, PNG grayscale images), using PyTorch's ImageFolder dataset class for automatic label assignment.

SI-6: MITRE ATT&CK for ICS JSON Reference, The reporting module shall read a locally stored JSON file containing the MITRE ATT&CK for ICS matrix (tactics and techniques), querying it by predicted malware family name to retrieve associated adversary behavior mappings.

SI-7: SQLite Database Interface, The event logging module shall interface with a local SQLite database file via Python's sqlite3 standard library, performing INSERT operations for new detection events and SELECT operations for dashboard history display.

2.8.3 Hardware Interfaces

HI-1: NVIDIA GPU via CUDA, The system shall interface with the host NVIDIA GPU through CUDA 13.1 for accelerated CNN training and inference. GPU availability shall be detected automatically via torch.cuda.is_available(); the system shall gracefully fall back to CPU processing if no compatible GPU is detected, with a non-blocking notification displayed to the user.

HI-2: Local File Storage, The system shall require at least 100 GB of local disk storage: approximately 5 GB for the Malimg and supplementary datasets, 10 GB for Docker container images, 2 GB for trained model weights, and the remainder for generated forensic reports and PCAP captures from the Digital Twin simulation.

HI-3: Network Interface Card, The Digital Twin simulation module requires access to the host NIC for Docker bridge network creation. No physical network interfaces shall be used for malware traffic; all simulation traffic shall remain within Docker virtual network bridges.

2.8.4 Communications Interfaces

CI-1: Modbus TCP (Industrial Protocol), The Digital Twin simulation shall generate and capture Modbus TCP traffic on port 502 between emulated IIoT nodes within the isolated Docker bridge network. Modbus function codes including Read Coils (0x01), Read Holding Registers (0x03), and Write Single Coil (0x05) shall be simulated.

CI-2: MQTT (IoT Messaging Protocol), The Digital Twin simulation shall deploy an MQTT broker (Mosquitto or equivalent) within a Docker container and generate publish/subscribe messaging traffic on port 1883 simulating IIoT sensor telemetry and command-and-control patterns.

CI-3: Streamlit HTTP Server, The dashboard shall be served by Streamlit's built-in Tornado HTTP server on localhost port 8501 (default) or a user-configured port. All dashboard communication between browser client and Streamlit server shall occur over HTTP on the local network interface only.

CI-4: Detection Event Database, All inter-process communication between the detection pipeline and the event logging database shall occur via Python's sqlite3 module using local file I/O. No network database connections shall be required.